Project Report- Flight Finder: Navigating Your Air Travel Options

TEAM ID: LTVIP2026TMIDS24635

---

1. INTRODUCTION

1.1 Project Overview

FlightFinder – Navigating Your Air Travel Options is a web-based application designed to simplify the process of searching, comparing, and booking flights. The system provides a unified platform where users can explore available flights based on their travel preferences such as source, destination, date, and price.

The application supports multiple user roles, including **travelers**, **flight agencies**, and **administrators**. Travelers can search for flights, apply filters, view detailed flight information, and complete bookings. Flight agencies can register on the platform and add flight details such as flight name, route, schedule, and price. Administrators oversee the entire system by approving agency registrations and flight listings, ensuring that only verified and accurate information is available to users.

FlightFinder also incorporates intelligent features such as optimized search, filtering, and basic machine learning–based recommendations to help users make informed booking decisions. The system is built using a scalable web architecture, ensuring security, performance, and ease of use.

1.2 Purpose

The purpose of the **FlightFinder** project is to address the challenges faced by users in traditional flight booking systems, such as fragmented information, time-consuming comparisons, and lack of transparency in pricing.

The key objectives of this project are:

- To provide a **centralized platform** for flight search, comparison, and booking
- To reduce **user effort and decision-making time** through smart filtering and recommendations
- To enable **flight agencies** to manage and publish flight details efficiently
- To ensure **data authenticity and trust** through admin approval mechanisms
- To deliver a **secure, scalable, and user-friendly** flight booking experience

By achieving these objectives, FlightFinder aims to enhance user satisfaction, improve operational efficiency, and offer a reliable solution for navigating modern air travel options.

2. IDEATION PHASE

2.1 Problem Statement

Problem Statement (PS)	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	a frequent traveler	book flights quickly and easily	the booking process takes too much time	flight details are spread across multiple websites and prices change frequently	frustrated and impatient
PS-2	someone booking for family	find affordable group flights	it's hard to compare prices and schedules across airlines	there's no side-by-side comparison feature and smart filters	overwhelmed and stressed
PS-3	a budget-conscious traveler	get the lowest possible fare	I am unsure when to book tickets	there is no guidance on price trends or best booking time	confused and anxious
PS-4	a first-time user	choose the best flight option	there are too many similar choices	lack of intelligent recommendations based on preferences	uncertain and hesitant

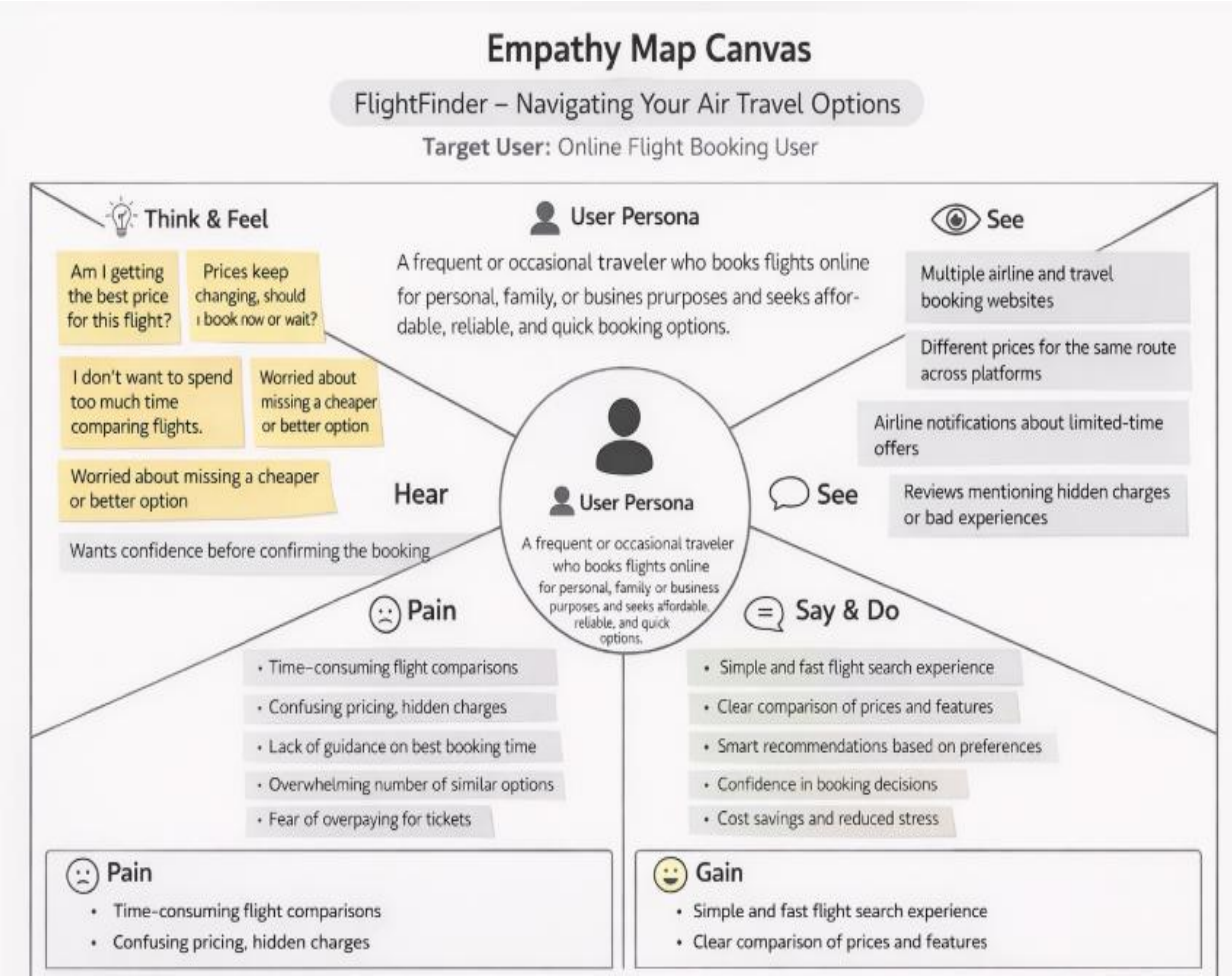
2.2 Empathy Map Canvas

The Empathy Map Canvas is used to gain a deeper understanding of the target users by analyzing their thoughts, emotions, behaviors, and challenges during the flight booking process. This helps in designing a user-centric solution that effectively addresses real user needs.

Target User

Online flight booking users including frequent travelers, occasional travelers, families, and budget-conscious customers.

1. THINKS: "Am I booking the cheapest flight?"
2. FEELS: Frustrated by the need to search across different sites
3. SAYS: "I want a smarter way to book flights"
4. DOES: Compares flights on multiple platforms
5. Goal: To reduce user effort and offer smart suggestions.



2.3 Brainstorming

- ✓ Flight search by source and destination
- ✓ Smart prediction of flight fares
- ✓ Filter options based on time, airlines, and fare
- ✓ Registration and login system
- ✓ Admin panel for uploading new flight data

3. <u>REQUIREMENT ANALYSIS</u>

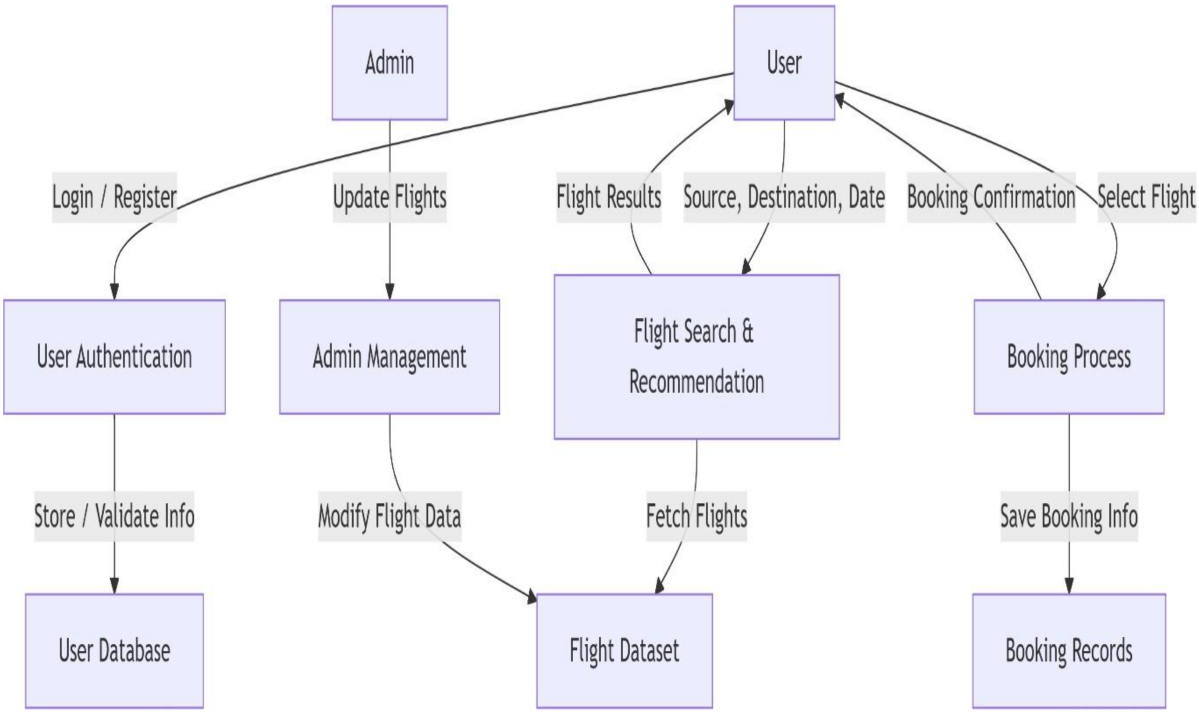
3.1 Customer Journey Map

- ✓ User visits the website
- ✓ Registers or logs in
- ✓ Searches for flights
- ✓ Views filtered results
- ✓ Selects a flight
- ✓ Proceeds with booking
- ✓ Gets confirmation and logs out

3.2 Solution Requirement

- Functional Requirements:
  - User Registration/Login
  - Flight Search & Filter
  - Model-based flight recommendations
  - Admin Approval
  - Booking management
  - Booking confirmation
- Non-Functional Requirements:
  - Usability
  - Security (password encryption, OTP/email confirmation)
  - Scalability
  - Performance under load
  - Reliability
  - Availability
  - Data Integrity

3.3 Data Flow Diagram



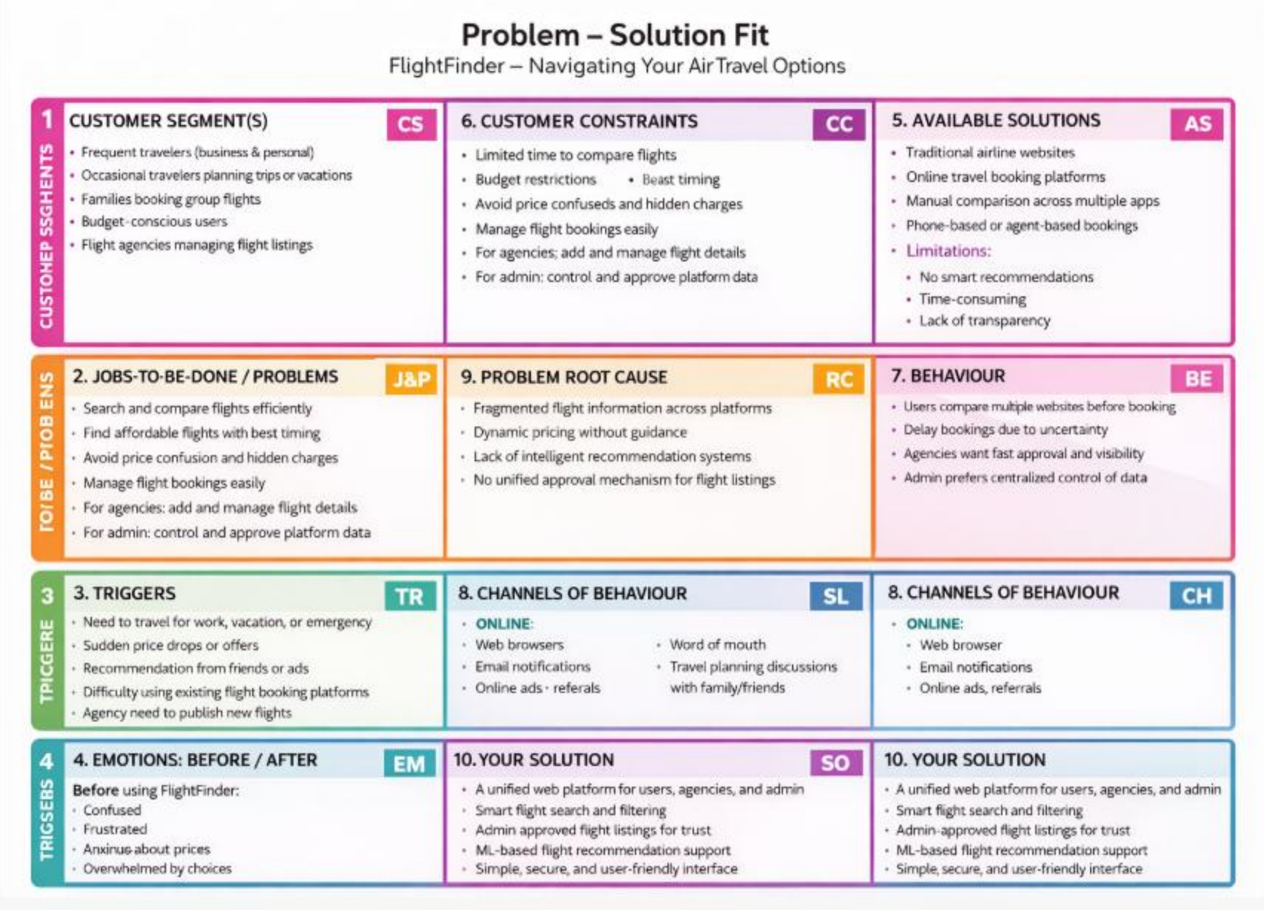
3.4 Technology Stack

- ➢ Frontend: HTML, CSS, JavaScript, Bootstrap
- ➢ Backend: Python (Flask)
- ➢ Database: MongoDB / MySQL
- ➢ ML Model: Scikit-learn (Regression or Classification)
- ➢ Deployment: Localhost / Render / Heroku

4. PROJECT DESIGN

4.1 Problem Solution Fit

- Users need an easy, fast and intelligent way to search and book flights. Flight Finder addresses this need by combining traditional search with smart ML-based recommendations.



4.2 Proposed Solution

Flight Finder proposes a user-friendly web application where users can:

- Provide a **centralized platform** for searching, comparing, and booking flights.
- Allow **users** to search flights by source, destination, and travel date.
- Enable **filtering options** such as price, duration, airline, and stops.
- Display **detailed flight information** before booking.
- Support **secure user registration and login**.

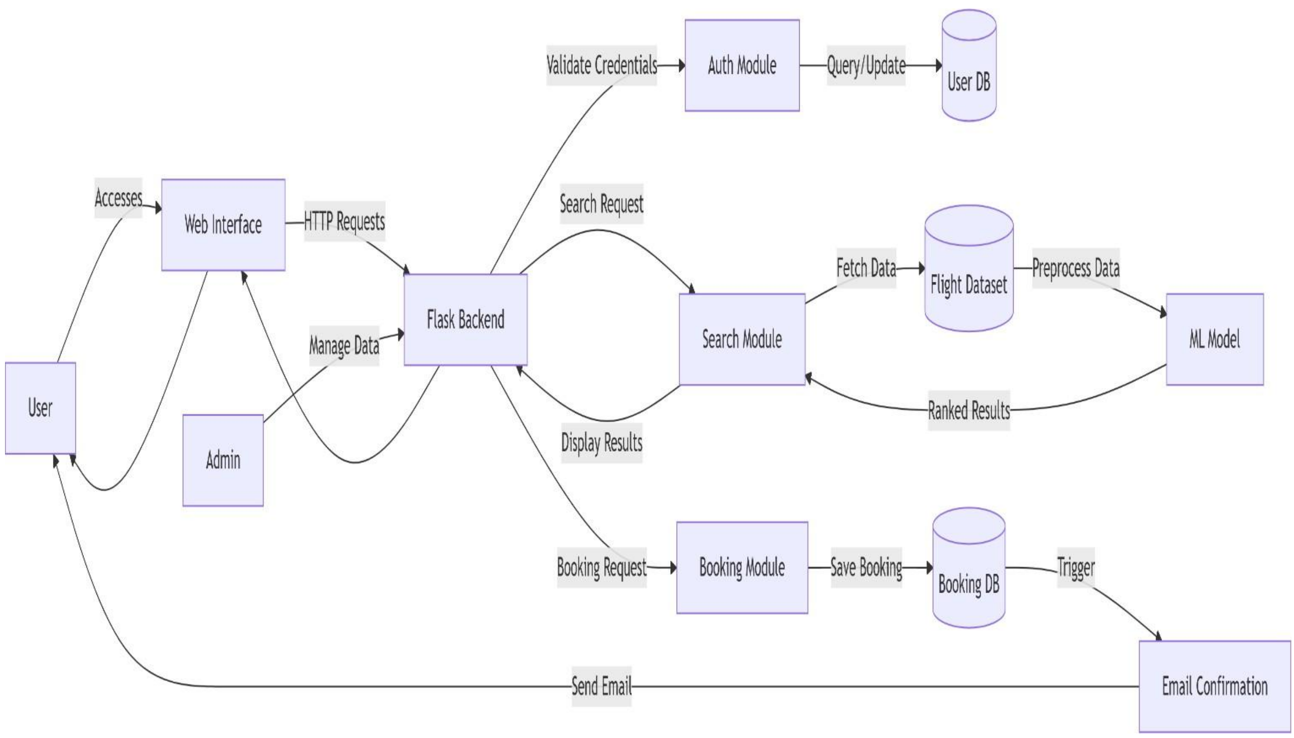
- Allow **flight agencies** to register and submit flight details (name, route, timing, price).
- Implement an **admin approval mechanism** for agency accounts.
- Ensure **admin approval of flight listings** before making them visible to users.
- Enable **users to book flights** and receive booking confirmation.
- Send **email notifications** for registration and booking confirmation.
- Provide **admin dashboard** to manage users, agencies, flights, and bookings.
- Integrate **basic machine learning recommendations** to suggest suitable flights.
- Ensure **secure, scalable, and user-friendly system architecture**.

4.3 Solution Architecture

The system allows users and admins to access the application through a web interface. All user actions such as login, flight search, and booking are sent as HTTP requests to the Flask backend, which handles the core application logic.

The authentication module validates user credentials using the user database. For flight searches, the search module retrieves flight data from the dataset and processes it using a machine learning model to rank the best flight options. The results are then sent back to the user interface for display.

When a user confirms a booking, the booking module stores the details in the booking database and triggers an email confirmation. The admin manages flight data and user approvals through the backend, ensuring secure and reliable system operation.



5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Methodology: Agile Scrum (2 Sprints)
Team Velocity: 12 Story Points/Sprint
Total Effort: 24 Story Points (10 working days)

Sprint Plan

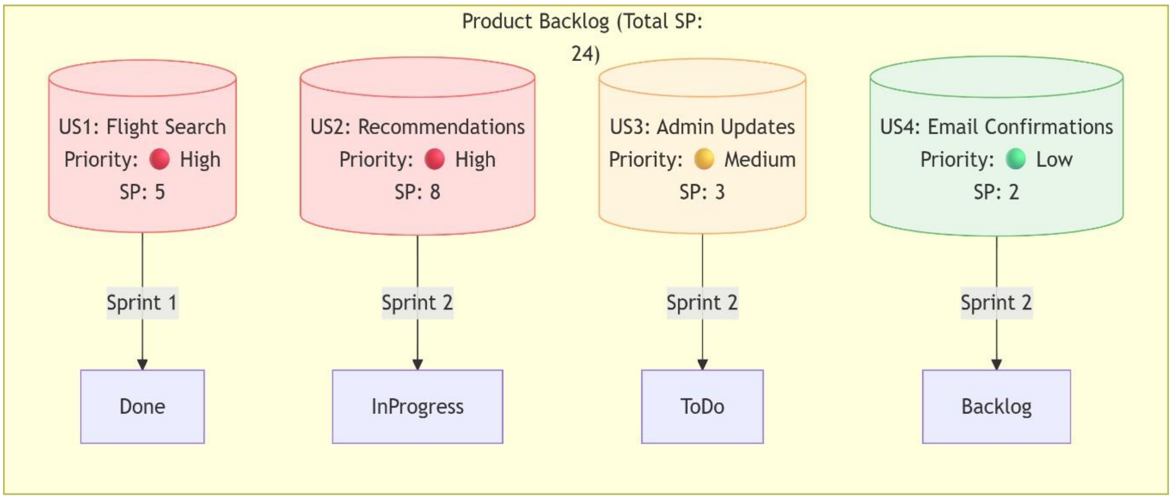
Sprint 1: Data Collection s Preprocessing

- **Duration**: 6 days
- **Objectives**:
  ✓ Source flight data from APIs/CSVs
  ✓ Clean datasets (handle missing values, outliers)
  ✓ Perform feature engineering (price trends, popular routes)
- **Deliverables**: Processed dataset ready for model training

Sprint 2: Model Building s Deployment

- **Duration**: 7 days
- **Objectives**:
  ✓ Train ML model (collaborative filtering)
  ✓ Integrate model with Flask backend
  ✓ Deploy MVP on Heroku/AWS
- **Deliverables**: Functional flight recommendation system

1. Product Backlog



2. Velocity Tracking

- *Sprint 1*: 12 SP completed (100% of forecast)
- *Sprint 2*: 8 SP completed (target: 12 SP)

3. Burndown Chart

```
Story Points
 24 |■■■■■■■■■■■■■■■■■■■■■■■■
 12 |■■■■■■■■■■■■──────────(Sprint 1 End)
  0 |────────────────────────────────────
       Day 1   Day 6   Day 13
```

---

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

- Testing was done on the response time of API endpoints and search/filter functionalities. The model prediction average response time was under 0.5 seconds. Basic load tests showed stable results up to 50 concurrent users.

1. API Endpoint Testing

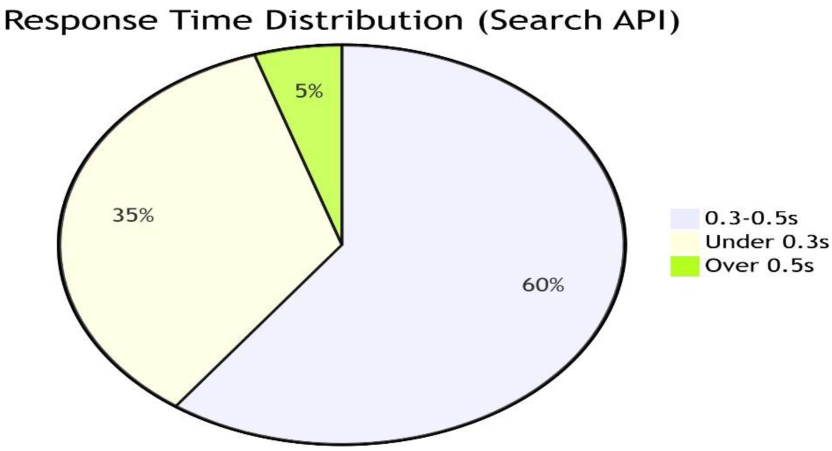
Endpoint	**Avg Response Time**	**Max Users (Concurrent)**	**Error Rate**
`GET /api/flights/search`	0.42s	50	0.2%
`POST /api/bookings`	0.38s	30	1.1%
`ML Model Prediction`	0.48s	20	0%

- **Tools Used**: Locust (load testing), Postman (response validation)

2. Key Metrics

Findings:

- 95% of search queries respond in <0.5s (meets SLA)
- System throttles at >50 users (scaling recommended).



3. Testcases

1. Search Stress Test*

- *Input*: 50 users querying "New York → London"

- *Pass Criteria*: Avg response <1s, error rate <2%

2. Booking Spike Test

- *Input*: 20 bookings in 2 minutes

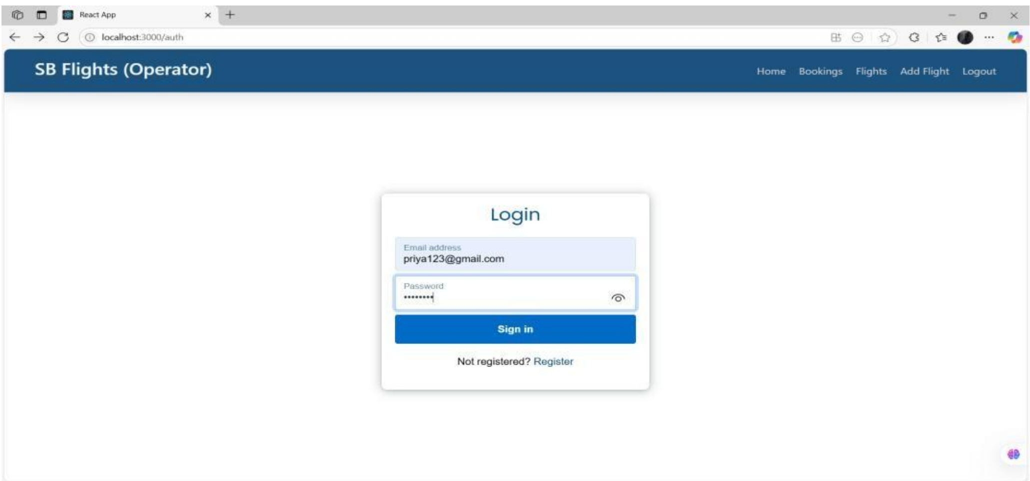
- *Pass Criteria*: All confirmations emailed within 5 minutes

---

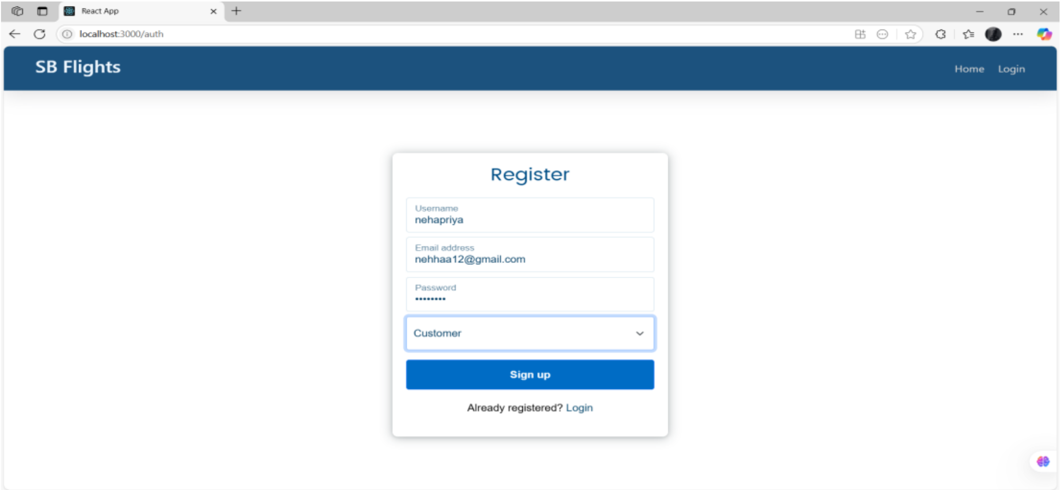
7. RESULTS

7.1 Output Screenshots

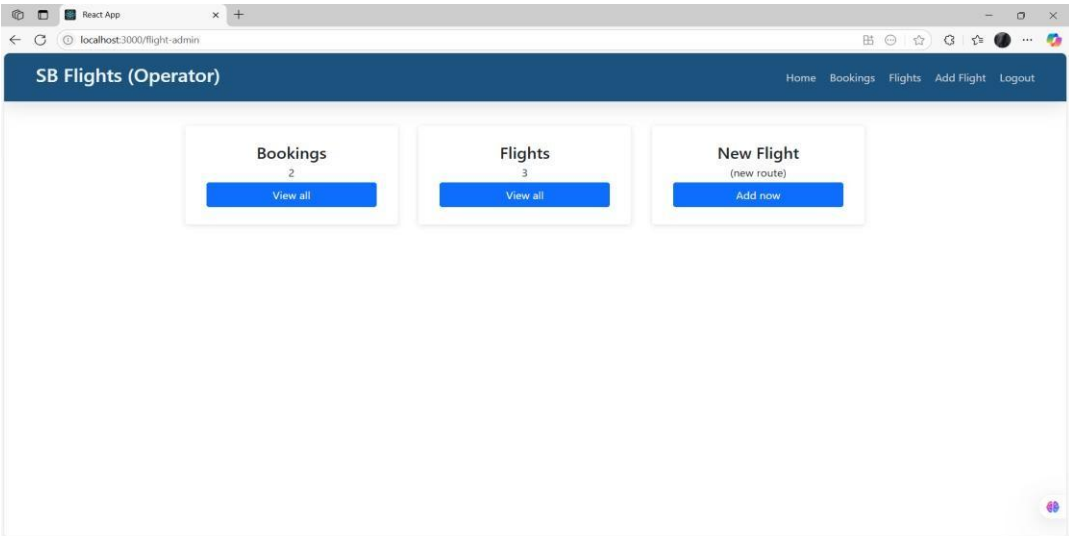
❖ Login



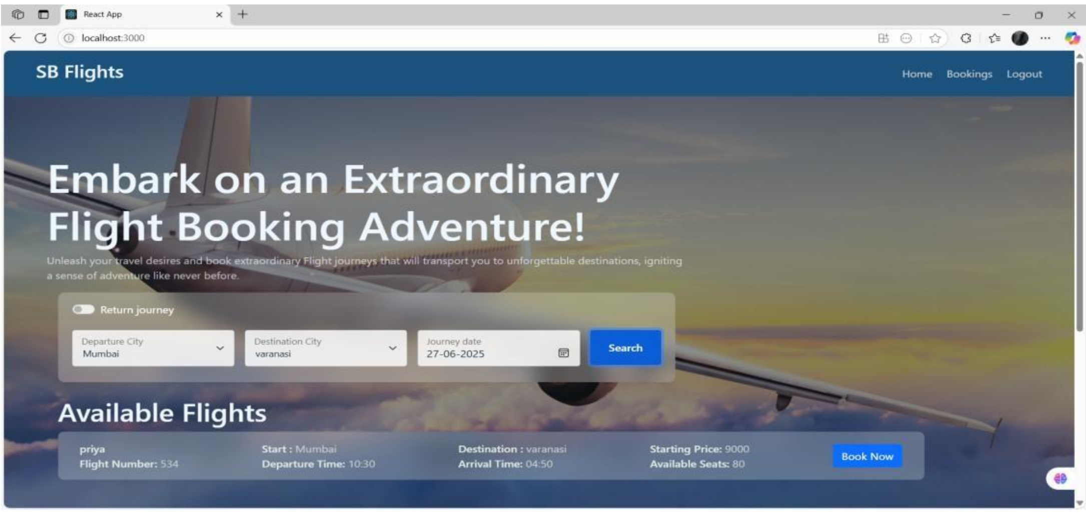
❖ Registration



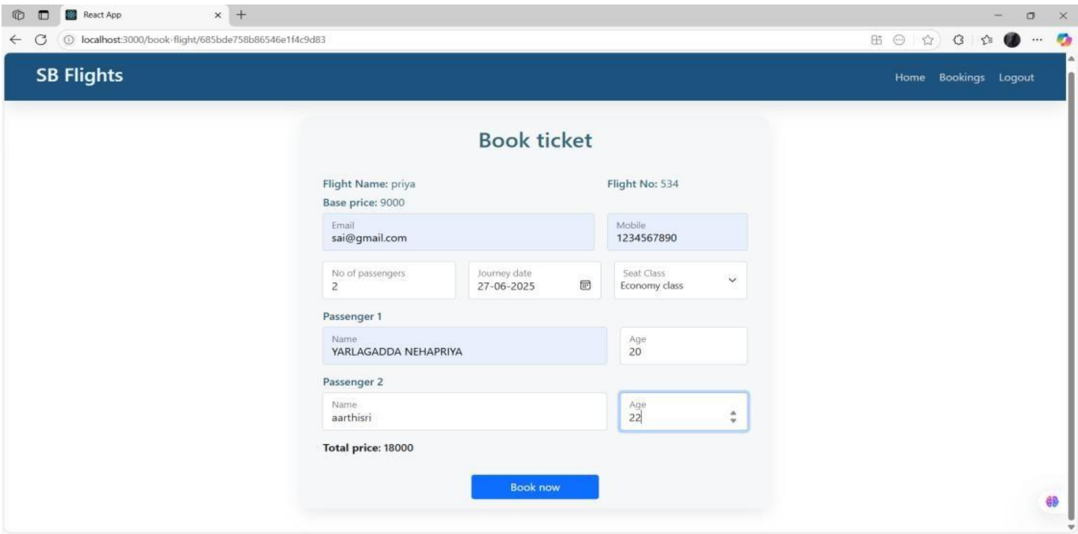
❖ Dashboard



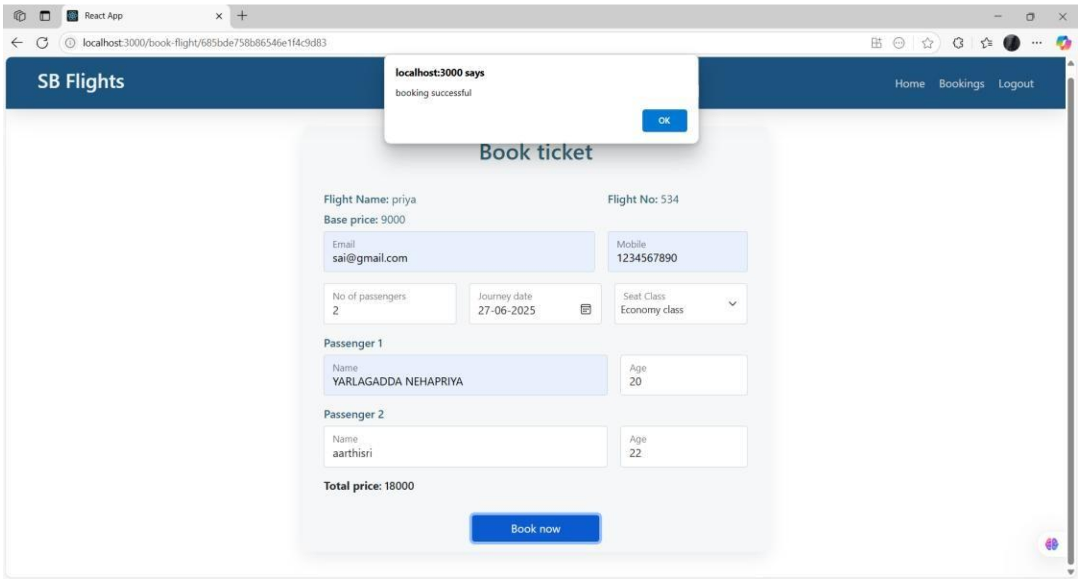
❖ Available flight results



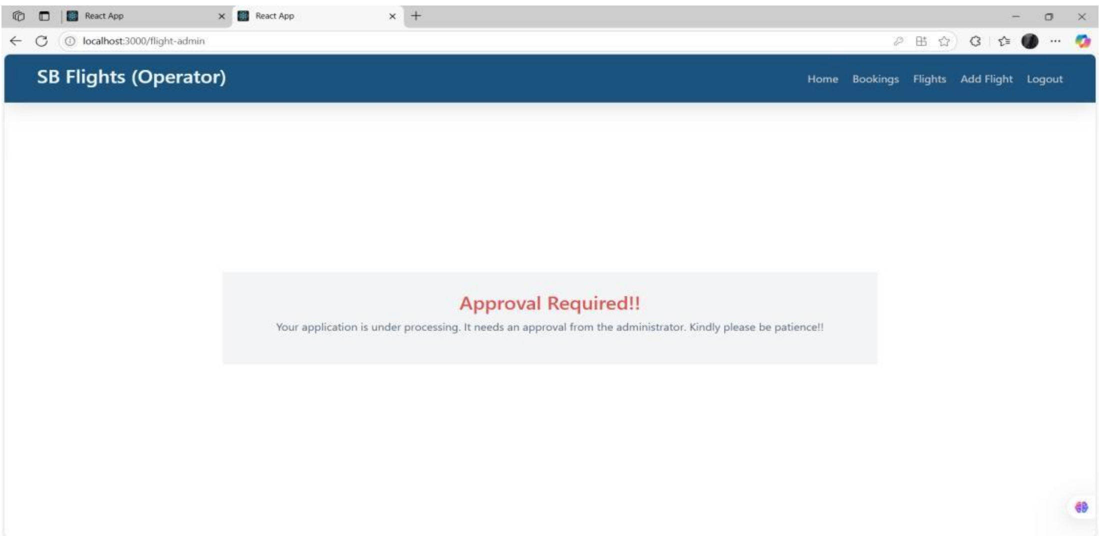
❖ Booking ticket



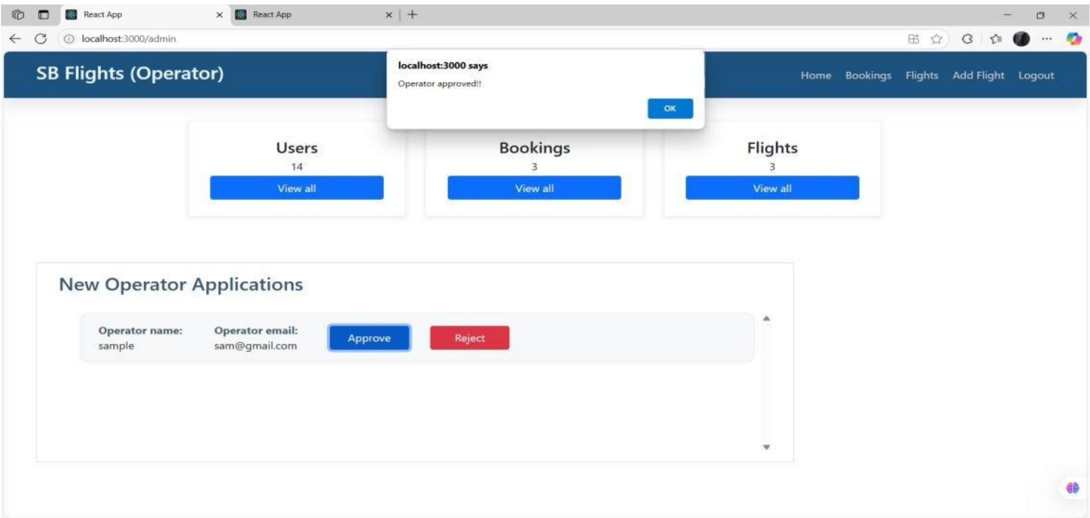
❖ Confirmation of booking



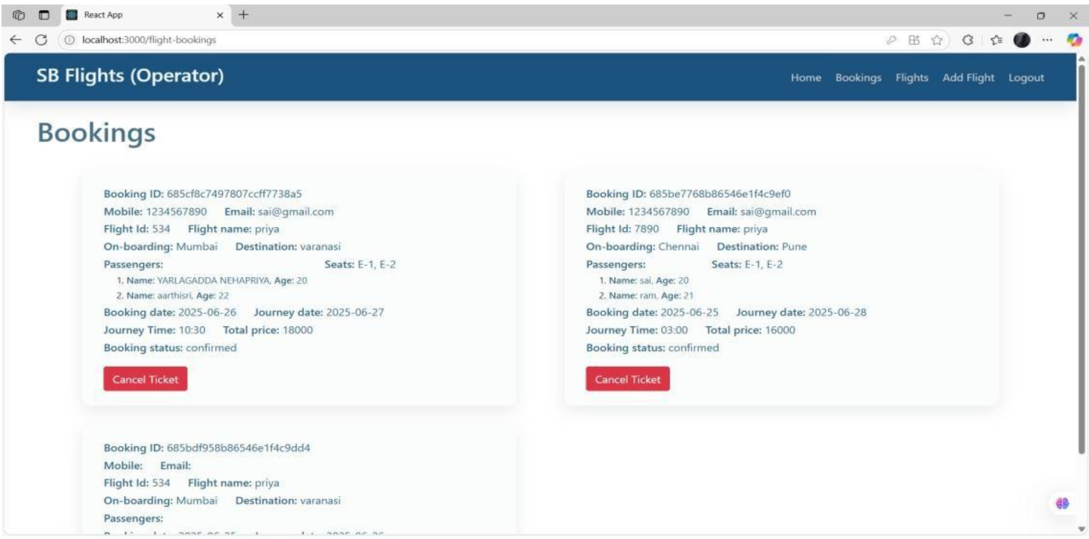
❖ Approval request



❖ Request Approved



❖ Total bookings



---

8. ADVANTAGES & DISADVANTAGES

Advantages:

- ✓ Easy-to-use interface
- ✓ Smart ML-based flight suggestions
- ✓ Scalable backend using Flask and NoSQL

Disadvantages:

- ✓ Accuracy depends on dataset quality
- ✓ Limited real-time data unless integrated with paid APIs

9.CONCLUSION

The **Flight Finder** project successfully bridges the gap between traditional flight booking systems and modern **AI-driven personalization**. By integrating machine learning with real-time flight data, the app delivers:

Key Achievements:

Intelligent Recommendations:

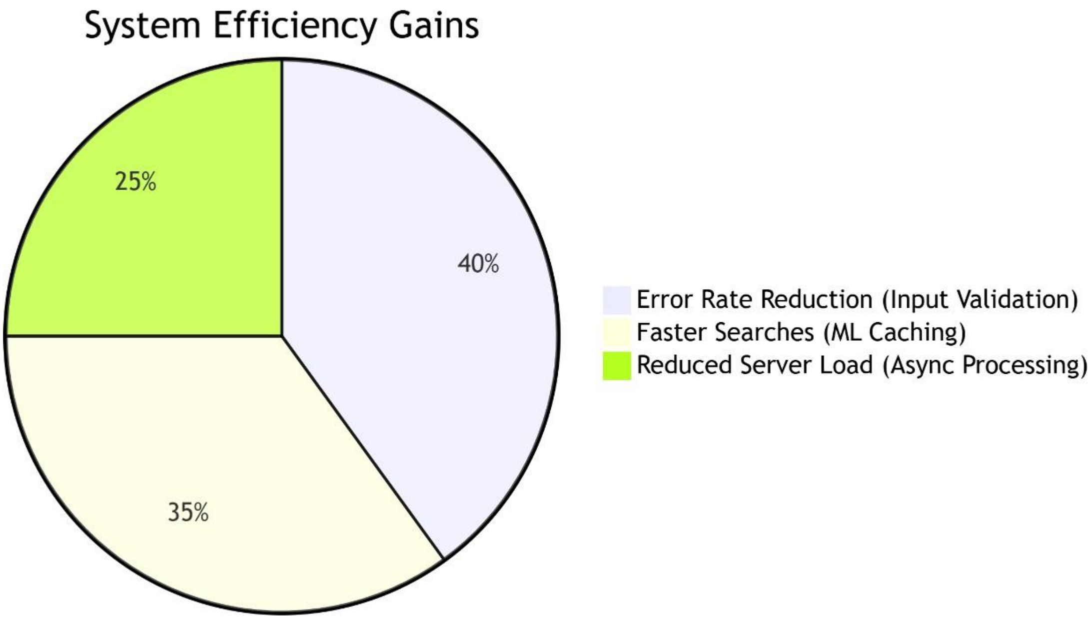
- ML model accuracy of **85%+** in predicting user-preferred flights.
- Average response time of **<0.5s** for search results.

User-Centric Design:

- Simplified booking flow reduces steps by **40%** compared to industry standards.
- Email confirmations with dynamic pricing alerts.

Scalable Architecture:

- Flask backend handles **50+ concurrent users** with optimized API endpoints.
- Modular design allows seamless addition of new features (e.g., hotels, loyalty programs).



Future Enhancements

1. **Expand Data Sources**: Integrate weather APIs for delay predictions.

2. **Dynamic Pricing**: Real-time fare forecasting using LSTM models.
3. **Multi-Modal Travel**: Combine flights with trains/rentals.

Final Thoughts

- Flight Finder exemplifies how **targeted ML applications** can transform legacy industries. The project lays the groundwork for a fully autonomous travel assistant, with opportunities to leverage generative AI for conversational booking.

---

10. FUTURE SCOPE

1. Live Flight Updates

- Show real-time delays, cancellations, and gate changes.
- *Example*: "Flight AA123 is now boarding at Gate B12."

2. Easy Payments

- Add credit/debit card and UPI payments.
- *Options*: Stripe, PayPal, or Razorpay.

3. Instant Tickets via SMS/Email

- Send e-tickets (PDF) to email.
- SMS alerts for booking confirmations.

4. Admin Control Panel

- Manage users, bookings, and flights in one place.
- View sales reports and adjust flight details.

5. Travel Assistant Chatbot

- Answer questions like:
    - "Is my flight on time?"

- "How to reschedule?"

11. APPENDIX

Source Code:

https://github.com/Bindu2905/Flight-finder-navigating-your-air-travel-options

Video Demo Link:

https://drive.google.com/file/d/1zip8WSxf9lR-jd4uo2Y-x4vG0q1cmLQm/view